



Applied Software Project Report

By

Makesh Balasubramaniam

**A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment
of the requirements for the degree of Master of Science in Computer Science**

May, 2026



Scaler Mentee Email ID : makeshb@ymail.com

Thesis Supervisor : Naman Bhalla

Date of Submission : 13/05/2026

© The project report of <Full Name of the Student> is approved, and it is acceptable in quality and form for publication electronically

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

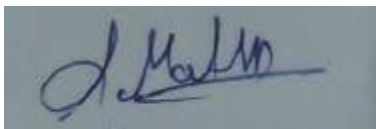
.....

Project Guide / Supervisor

DECLARATION

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 05-03-2024 to 19-06-2024 , is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Makesh Balasubramaniam

A handwritten signature in blue ink, appearing to read 'Makesh', is shown on a light blue background.

Date: 13/05/2026

ACKNOWLEDGMENT

I would like to express my deepest gratitude to everyone who has been a part of this incredible journey. First and foremost, to my family — your unwavering support, patience, and encouragement through every late night and challenging moment made this achievement possible. Words cannot fully capture how much your belief in me has meant.

To the Scaler instructors and mentors — thank you for your dedication, expertise, and the passion you brought to every session. Your ability to break down complex concepts and inspire curiosity has shaped not just my technical skills, but my entire approach to problem-solving. A special thank you to the founders of Scaler for building a platform that genuinely transforms careers and nurtures talent with purpose and vision.

To my batch members — this journey was made richer because of the camaraderie, and the energy we brought to every discussion, and challenge together. Your enthusiasm was contagious and kept me going on the toughest days.

To my seniors and colleagues — thank you for the knowledge you shared so generously, the guidance you offered without hesitation, and the culture of learning you helped foster. Every conversation, code review, and piece of advice has contributed to who I am as a professional today.

Earning this Master's degree is not just a personal milestone — it is a shared achievement with each one of you. Thank you from the bottom of my heart.

Table of Contents

List of Tables

Table No.	Title	Page No.
1	Notifications	27
2	Accounts	29
3	CIF	32
4	Customers	34
5	KYC_DETAILS	36
6	user_roles	38
7	users	38

List of Figures

Figure No.	Title	Page No.
1	Use case Diagram	16
2	Architect Diagram	17
3	Notifications-service Class Diagram	28
4	User Registration Service Class Diagram	30
5	Payment-service Class Diagram	41
6	CI/CD Diagram	46
7	Deployment Diagram	47

Table of Contents

List of Tables	6
List of Figures.....	7
Applied Software Project	9
Abstract.....	9
Project Description	10
System Architecture & Key Components	11
Use case Diagram:.....	17
Architect Diagram:	18
Project Development Process.....	18
PHASE 1: DEFINITION	18
PHASE 2: PLANNING	20
PHASE 3: DEVELOPMENT (3 Sprints).....	22
Requirement Gathering:	28
Notifications-service:	28
User Registration Service:	31
Payments-service:	41
Deployment Flow	45
CI/CD:.....	46
Deployment:.....	48
Technologies Used	51
Conclusion:.....	53
References:	56
Functional References:.....	56
Technical References:.....	56
Tools used:.....	57

Applied Software Project

Abstract

This project presents a comprehensive digital banking platform designed to revolutionize banking accessibility for retail customers through a secure, 24/7 online portal. The system addresses the growing demand for seamless digital financial services by enabling end-to-end account creation, verification, and transaction management without physical branch visits.

The platform implements a robust online registration and Know Your Customer (KYC) verification process leveraging India's national identity infrastructure—PAN (Permanent Account Number) and Aadhaar—ensuring regulatory compliance with Reserve Bank of India (RBI) guidelines while maintaining stringent security standards. Employees from partner

organizations benefit from streamlined onboarding procedures, facilitating faster financial inclusion and banking access.

Key functionalities include comprehensive Transaction management, enabling users to register and verify payees for secure fund transfers. The system supports both domestic payments through NEFT, RTGS, and IMPS integration with the National Payments Corporation of India (NPCI), as well as international remittances through SWIFT corridors. Real-time transaction processing ensures immediate fund availability and confirmation, enhancing user experience and operational efficiency.

Built on a secure, scalable architecture with multi-layer authentication and encryption protocols, the platform operates continuously across geographical regions, providing uninterrupted banking services. This accessibility eliminates traditional banking hour constraints and geographical barriers, particularly benefiting working professionals, rural populations, and users requiring after-hours banking services.

The application demonstrates significant potential for financial institutions seeking digital transformation, reducing operational costs associated with physical infrastructure while expanding customer reach. By automating account opening, KYC verification, and payment processing workflows, the platform exemplifies how modern software solutions can democratize banking services, improve transaction efficiency, and align with India's vision of a digitally empowered financial ecosystem.

.

Project Description

This project delivers a secure, scalable, and always-available digital banking platform tailored for retail customers. It enables seamless online account creation, KYC verification, and transaction management, eliminating the need for physical branch visits. The platform leverages India's national identity infrastructure (PAN and Aadhaar) for regulatory compliance and robust security, aligning with RBI guidelines. It supports both domestic and international payments, real-time transaction processing, and multi-layer

authentication, making banking accessible to all user segments, including rural and working professionals. This acts as an channel between UI/Mobile interface and Core banking System which eliminates the need for physical branch visits.

This Project has 3 microservices

- notifications-service
- payments-service
- useraccounts-service

Considering the Project scope and timeline I have limited the functionality only to a Minimum Viable Product (MVP) which supports the users need to user Registration, Account generation,

Payment initiation and send notification, As this project grows and goes through multiple user's feedback, we keep adding new features in future, like processing the payment in real time by integrating with core banking system in synchronous way(Currently the payments accepts are pulled by core banking from payments table and processed) and

Integrating notification service to cloud service like AWS SNS.(Currently the third-party service pulls the notification data from Notification table and publishes) and adding alerts and observability through Prometheus and Grafana .

System Architecture & Key Components

- **Client (C):** The end-user interface (web/mobile app) for registration, authentication, and banking operations.
- **User Accounts-Service (UAS):** Handles user authentication, registration, KYC, and account management and Token Generation.

- **Payments-Service (PS):** Manages payment initiation, accepts the payment request and generate transaction records and stores in DB which will be process by Core Banking application.
- **Notifications-Service (NS):** accepts notifications for registration, status updates, and payments and stores in DB, which will be pull be externa system and published.
- **Databases:** Each service has a dedicated database (UASDB, PSDB, NSDB) for secure data storage.
- **Authentication:** All the API's in all microservice's are secured by JWT Token authorization.

API Details & Flow:

User Accounts-Service (UAS):

Request Initiated By Admin:

1. User Registration (Authentication)

- **Endpoint:** POST /auth/register
- **Request:** { userId, pwd, roles }
- **Flow:**
 - Client sends registration details ie, initiated by ADMIN.
 - UAS stores credentials and roles in UASDB.
 - The stored details are further used to generate the token by client using user credentials while accessing the API's.
 - Success response sent to client.

2. User Login & Token Generation

- **Endpoint:** POST /auth/login
- **Request:** { userId, pwd }
- **Flow:**
 - Client submits credentials , initiated by ADMIN

- UAS validates against UASDB.
- On success, returns JWT token for session authentication which has an expiry date.

Request Initiated By End User:

1. Authenticated User Registration (KYC & Account Creation)

- **Endpoint:** POST ap1/v1/users/register
- **Headers: Authorization:** Bearer <JWT Token>
- **Request:** { Personal Details, Contact Details, KYC Details (PAN, Aadhaar) }
- **Flow:**
 - JWT token validated.
 - UAS creates customer profile, CIF, and account with status **PENDING** in UASDB.
 - Sends registration notification to notification-service.
 - Returns customer number to client.

2. Registration Status Inquiry

- **Endpoint:** GET ap1/v1/users/pending/{customerNumber}
- **Headers: Authorization:** Bearer <JWT Token>
- **Flow:**
 - JWT token validated.
 - UAS fetches registration status from UASDB.
 - Returns status to client.

Request Initiated By Admin:

1. Admin Status Update (Account Activation)

- **Endpoint:** PUT ap1/v1/users/update
- **Headers: Authorization:** Bearer <JWT Token>
- **Request:** { customerNumber, status: "ACTIVE" }

- **Flow:**
 - JWT token validated.
 - UAS updates customer status in UASDB.
 - Sends status update notification to notification-service.
 - Returns update success response.

Payments-Service (PS):

Request Initiated By End User:

1. Payment Initiation & Processing

- **Endpoint:** POST `ap1/v1/payments/createPayment`
- **Headers:** Authorization: Bearer <JWT Token>
- **Request:** { FromAccount, ToAccount, Amount }
- **Flow:**
 - JWT token validated.
 - Accepts the payment request and stores transaction in PSDB.
 - Sends payment notification to notification service.
 - Returns payment success/Failure response with generated transaction id.
 - This transaction records will be pulled by core banking system for processing.

Notifications-Service (NS):

- **Endpoint:** POST `api/v1/notifications/send`
- **Headers:** Authorization: Bearer <JWT Token>
- **Request:** { type, recipient, subject }
- **Flow:**
 - JWT token validated.

- Accepts the notification request and stores transaction in NSDB.
- Returns payment success/Failure response.
- The notification details will be pulled by external system and published to user.

Sequence Flow Summary

1. **User Registration:** Client registers and receives credentials.
2. **Login:** Client logs in and receives JWT token.
3. **KYC & Account Creation:** Authenticated client submits KYC; account is created with pending status.
4. **Status Inquiry:** Client checks registration/account status.
5. **Admin Activation:** Admin updates account status to active; notification sent.
6. **Payments:** Authenticated client initiates payments; processed and notified in real-time.

Security & Compliance

- **Authentication:** JWT-based, multi-layered.
- **KYC:** PAN and Aadhaar integration for regulatory compliance.
- **Encryption:** All sensitive data and communications are encrypted.
- **Notifications:** All critical actions trigger notifications for transparency.

Key Benefits

- **24/7 Accessibility:** No branch or time constraints.
- **Regulatory Compliance:** Adheres to RBI and national KYC norms.
- **Operational Efficiency:** Reduces manual intervention and physical infrastructure costs.
- **Scalability:** Modular microservices architecture supports growth and new features.
- **Financial Inclusion:** Empowers rural and working populations with digital banking.

When a user wants to avail banking Service without visiting bank branch, then user register's

using Mobile client or Web Application which connects to our channel ie, banking Application services which was developed based on Spring boot microservices helps user in Account creation by Providing the necessary APIs for user registration, Token Generation, initiating payment and also creating notifications.

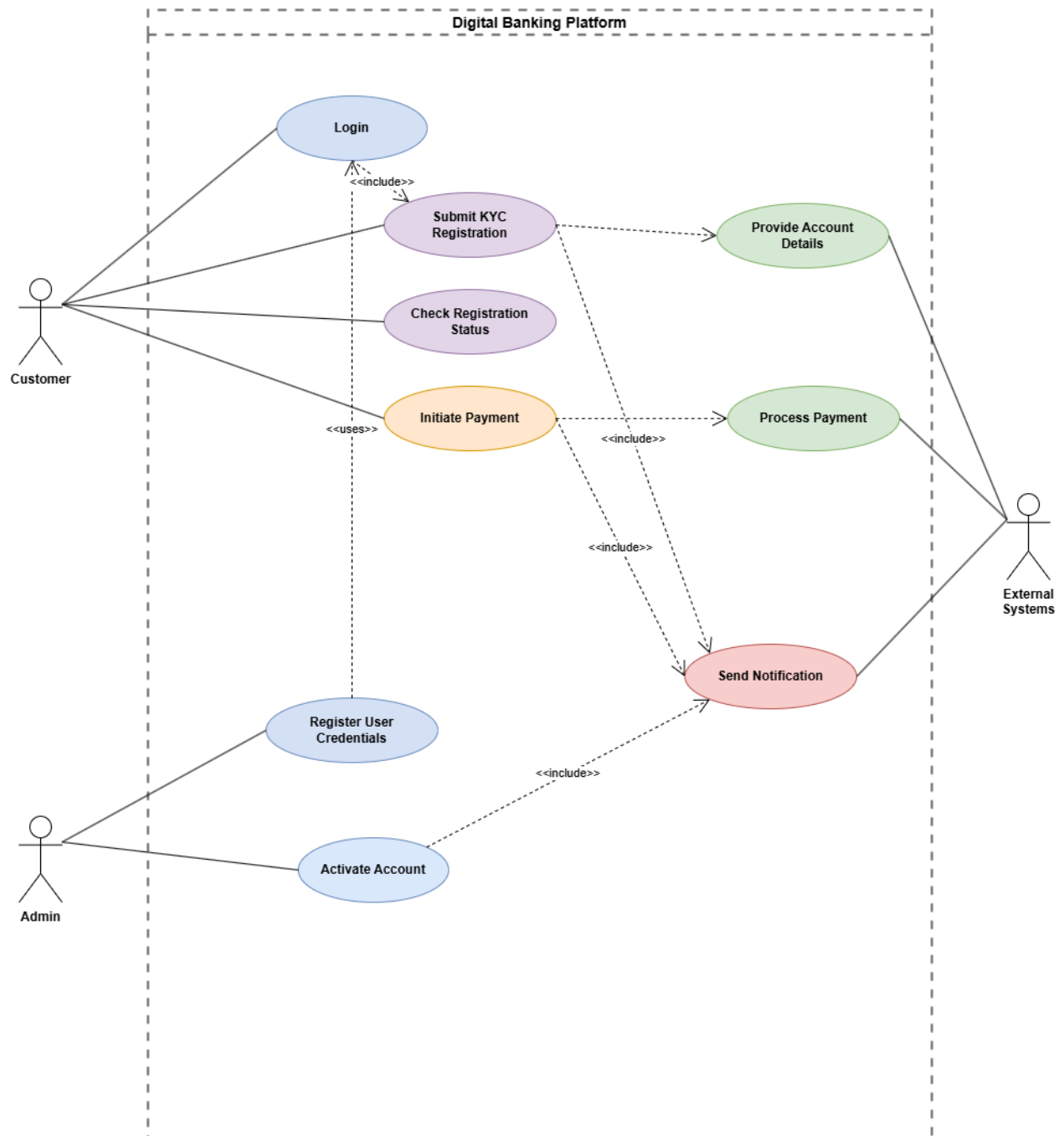
Admin will create user credentials for token generation , Also Admin can verify and approve the users for registration process ,so that accounts are enabled for users and post that user can initiate the Payments.

Here the Assumptions are a client ie , Web Application UI will be interacting with microservice's which we build and also, we have other Application like core banking Application which pulls

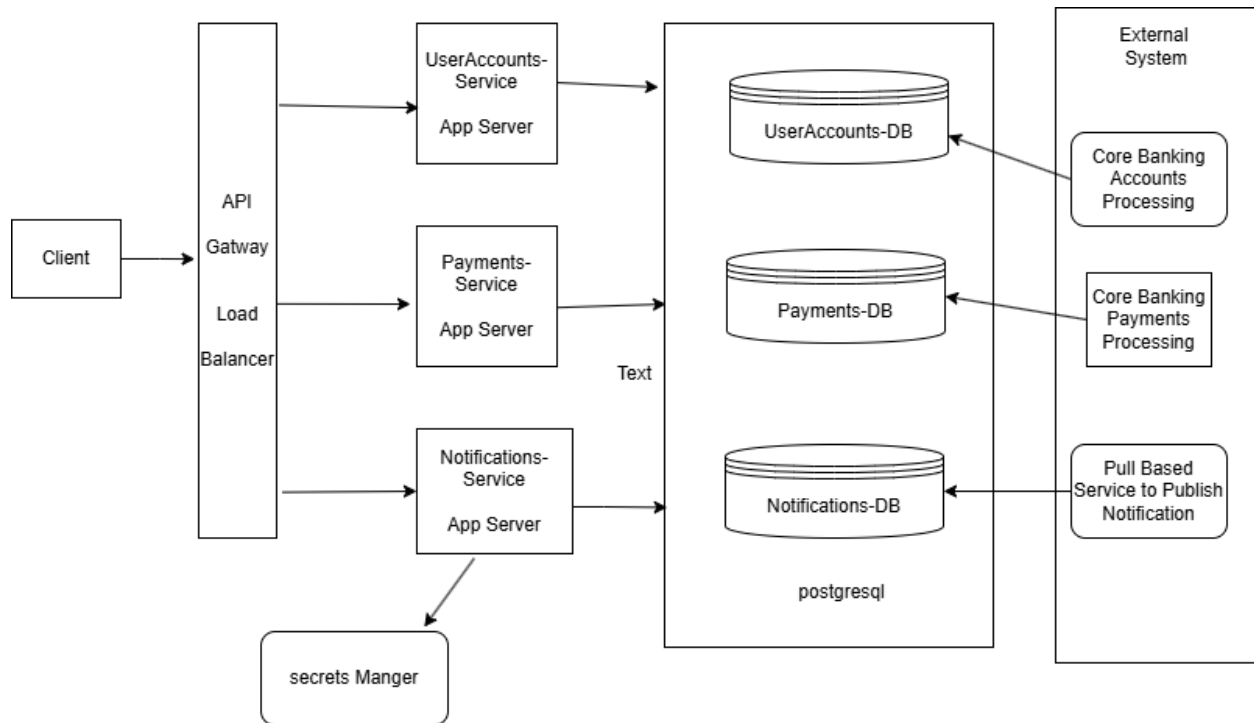
the customer records and maintains in it and also process payment records which is initiated from payment microservice, and also we have external message publishing system which pulls

the notification records from Notification service DB and publish to customers.

Use case Diagram:



Architect Diagram:



Project Development Process

PHASE 1: DEFINITION

Project Theme Selection

Theme: Secure Digital Banking Platform for retail customers

Problem: Need for 24/7 banking without physical branch visits, KYC verification, and secure online transactions

Solution: Microservices-based platform with 3 services: User Accounts, Payments, and Notifications

Design Sprint - Study Based on Personas

Persona	Needs	Features
Rural Customer	Easy account opening without branch visit	Mobile UI, Aadhaar KYC
Working Professional	24/7 payments and transactions	Always-available platform
Bank Admin	Efficient KYC verification	Admin dashboard

PHASE 2: PLANNING

Project Idea Presentation

MVP Scope:

- User registration and authentication (JWT)
- KYC-based account creation (PAN, Aadhaar)
- Payment initiation and transaction recording
- Notification generation and storage

Product Backlog

Feature	Priority	Sprint
Admin registers user credentials	High	Sprint 1
Login & Token Generation	High	Sprint 1
Send Notification	High	Sprint 1
Get Notification by User Id	High	Sprint 1
Customer submits KYC and creates account	High	Sprint 2
Customer checks registration status	Medium	Sprint 2
Admin activates customer account	High	Sprint 2
Customer initiates payment	High	Sprint 3
Get Payment Status by Notification id	Medium	Sprint 3
Review and Test end to end Product	High	Sprint 3

Development Environment Setup

- Java 21, Spring Boot 3.2.x, Maven
- MySQL 8.0 (UASDB, PSDB, NSDB)/ H2 DB for Testing

- Spring Security + JWT
- Postman for API testing
- Git/GitHub for version control

PHASE 3: DEVELOPMENT (3 Sprints)

Sprint 1 (2 weeks): Notifications & Authentication

Sprint Goal: Working on building of user Authentication API's and Notifications service.

Week 1: User Registration for Authentication and Token Generation API

Goal: Working with User Registration and Token Generation for securing API's

API/Feature	Endpoint	Implementation
User Registration	POST /auth/register	Store credentials in UASDB
Login & Token Generation	POST /auth/login	Validate & return JWT

Deliverables:

- User Accounts Service User Registration and Token Generation for JWT authentication
- Database tables: User Info, User Roles.
- Unit tests for User Registration and Token Generation.

Sprint Review: Demo User Registration and Token Generation.

Week 2: Notification Service with Authentication

API/Feature	Endpoint	Implementation
Send Notification	POST /api/v1/notifications/send	Store notification in NSDB

Get Notification by User Id	GET /getUnsendNotificationById/{userId}	Get stored Notification in NSDB
--------------------------------	--	------------------------------------

Deliverables:

- Notifications Service microservice
- Notification triggers from all major operations
- Complete API documentation (Swagger)

Sprint Review: Demo complete Notifications with JWT authentication.

Sprint 2 (2 weeks): KYC Registration and Approval with Authentication and notification

Sprint Goal : KYC-based account creation and Approval ,integration with Notification service

Week 1:

Goal: Working with KYC-based account creation, integration with Notification service and checking registration status

API/Feature	Endpoint	Implementation
KYC Registration	POST /api/v1/users/register	Create account with PENDING status
Check Registration Status	GET /api/v1/users/pending/{customerNumber}	Fetch status from UASDB

Deliverables:

- User Accounts Service with JWT authentication
- Database tables: User, Customer, Account
- Unit tests for KYC Registration and check Registration Status

Sprint Review: Demo KYC Registration , account creation and check Registration Status

Week2:

Goal: Enable account activation and integration with Notification service and test End to end flow whatever developed in Sprint 1 and Sprint2.

API/Feature	Endpoint	Implementation
-------------	----------	----------------

Account Activation	PUT /api/v1/users/update	Update to ACTIVE, trigger notification
--------------------	--------------------------	--

Deliverables:

- Admin workflow for account activation
- Integration with Notification service
- Integration tests for end-to-end flow

Sprint Review: Demo complete User Account registration flow, Approval and notification Process.

Sprint 3 : Payment Service with end-to-end flow.

Sprint Goal :

Payment Initiation, integration with user Account service and notification service and End to end Product Testing, review and Demo.

Week 1:

Goal : Payment Initiation , integration with user Account service and notification service

API/Feature	Endpoint	Implementation
Payment Initiation	POST /api/v1/payments/createPayment	Store transaction in PSDB, Integration for Debit account balance validation and notification
Get Payment Status by Notification id	GET /api/v1/payments /status/{transactionId}	Gets payment status by transactionId

Deliverables:

- Payments microservice with API to initiate payment and get status by Transaction id.
- Integration with user Accounts API and Notification service.
- Integration tests for end-to-end flow.

Sprint Review: Demo complete registration-to-payment flow

Week 2:

Goal : Adding Swagger for Payments Service, End to end review and testing of complete platform with all services working.

API/Feature	Endpoint	Implementation
End to end review and Testing of services integrated	All	End to end review and Testing of services integrated

Deliverables:

- Review code and deliverables with all services integrated and tested end-to-end.

Sprint Review: Demo complete platform with all services working

PHASE 4: DELIVERY

Project Result Presentation

Final Demo:

- Complete user journey: Registration → KYC → Activation → Payment
- Show all 3 microservices working together
- Demonstrate JWT security and role-based access
- Show notification triggers for each operation
- Explain external system integration points

Reflection on the Creation Process

What Went Well:

- Successfully implemented microservices architecture
- JWT authentication working across all services
- Met all MVP requirements within timeline

Challenges Faced:

- User registration and update flow with Multiple entities
- Inter-service communication complexity
- Database transaction management across services
- PAN and Aadhaar validation logic

Lessons Learned:

- Importance of API documentation from day one
- Value of incremental testing during development
- Benefits of Agile sprints for managing scope

Delivery of Experience Report

Technical Documentation:

- Architecture diagram showing 3 microservices
- API documentation with Swagger
- Database schemas for UASDB, PSDB, NSDB
- Sequence diagrams for key workflows
- Deployment guide and setup instructions

Code Repository:

- GitHub repository with all source code
- README with project overview and setup

- Unit and integration test cases

Future Enhancements:

- Real-time payment processing
- AWS SNS integration for notification publishing
- Admin dashboard UI
- Beneficiary Maintenance.

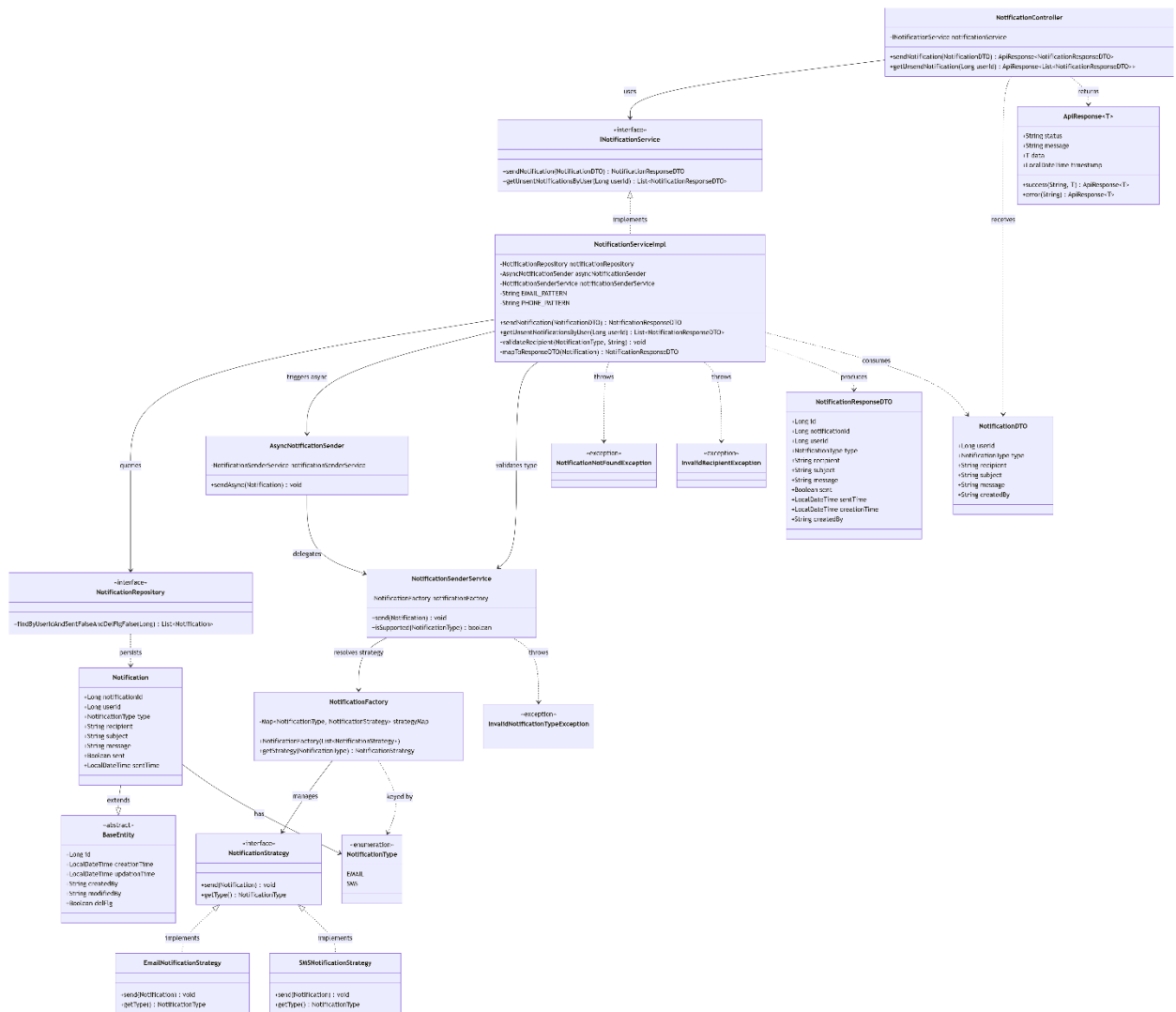
Requirement Gathering:**Notifications-service:**

This Notification Micro service is used to accept the notification of type SMS and EMAIL for payment, user creation and user Approval and store it in notification DB.

Assumption:

External messaging system will pull this and publish the notification to user

Class Diagram:



DB Schema:

create table notifications (

id bigint generated by default as identity,

created_by varchar(100) not null,

creation_time timestamp(6) not null,

del_flg boolean not null,

```
modification_time timestamp(6),
modified_by varchar(100),
updation_time timestamp(6),
message TEXT not null,
notification_id bigint,
recipient varchar(255) not null,
sent boolean not null,
sent_time timestamp(6),
subject varchar(500),
type varchar(10) not null check (type in ('EMAIL','SMS')),
user_id bigint not null,
primary key (id)
)
```

DB Specifications:

- id
- created_by
- creation_time
- del_flg
- modification_time
- modified_by
- updation_time
- message
- notification_id
- recipient
- sent
- sent_time
- subject
- type
- user_id

- Primary Key(id)

User Registration Service:

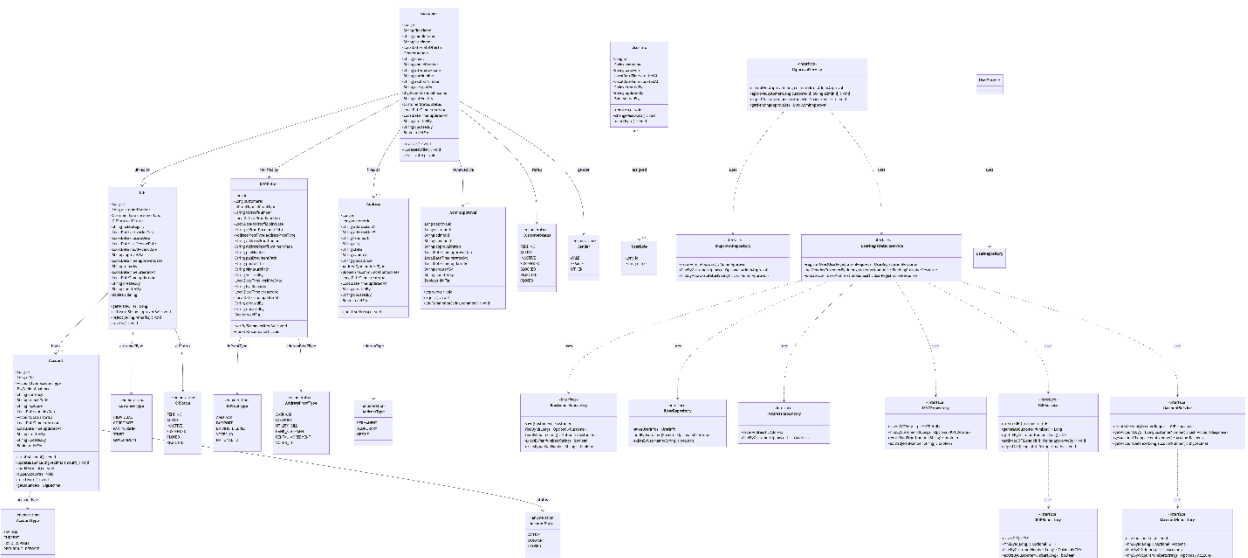
Admin can Register for user credentials and Later that same credentials will be used by end user to generate token for all API used.

End user inputs all personal details and register for Account creation which will be in pending status , once registration is completed user will get a notification .

Admin can view the status of Registration using customer Number and review it and approve the User registration, post that user will get a notification.

Once account is activated user will be enabled for initiating payments.

Class Diagram:



DB-Schema:

Accounts

Specifications:

- id
- created_at
- created_by
- del_flg
- updated_at
- updated_by
- account_type
- balance
- branch_code
- currency
- ifsc_code
- opening_date
- status
- cif_id
- Primary Key(id)

DB Schema

```
create table accounts (  
    id bigint generated by default as identity,  
    created_at timestamp(6) not null,  
    created_by varchar(255),  
    del_flg boolean,  
    updated_at timestamp(6),  
    updated_by varchar(255),  
    account_type varchar(255) not null,  
    balance numeric(38,2) not null,  
    branch_code varchar(10),
```



```
currency varchar(3),
ifsc_code varchar(11),
opening_date date,
status varchar(255),
cif_id bigint not null,
primary key (id)
)
```

Addresses:

Specifications:

- id
- created_at
- created_by
- del_flg
- updated_at
- updated_by
- address_line1
- address_line2
- address_type
- city
- country
- is_communication_address
- landmark
- postal_code
- state
- customer_id
- Primary Key(id)

DB Schema

```

create table addresses (
    id bigint generated by default as identity,
    created_at timestamp(6) not null,
    created_by varchar(255),
    del_flg boolean,
    updated_at timestamp(6),
    updated_by varchar(255),
    address_line1 varchar(255),
    address_line2 varchar(255),
    address_type          varchar(255)          check          (address_type          in
('PERMANENT','TEMPORARY','OFFICE')),
    city varchar(255),
    country varchar(255),
    is_communication_address boolean,
    landmark varchar(255),
    postal_code varchar(255),
    state varchar(255),
    customer_id bigint,
    primary key (id)
)

```

CIF:

Specifications:

- id
- created_at
- created_by
- del_flg

- updated_at
- updated_by
- activation_date
- approved_by
- approved_date
- cif_status
- closure_date
- customer_number
- customer_type
- last_review_date
- next_review_date
- remarks
- risk_category
- customer_id
- Primary Key(id)

DB Schema:

```
create table cif (
    id bigint generated by default as identity,
    created_at timestamp(6) not null,
    created_by varchar(255),
    del_flg boolean,
    updated_at timestamp(6),
    updated_by varchar(255),
    activation_date date,
    approved_by varchar(255),
    approved_date date,
    cif_status          varchar(255)          check          (cif_status          in
('PENDING','ACTIVE','INACTIVE','SUSPENDED','CLOSED','REJECTED')),
    closure_date date,
    customer_number bigint not null,
```

```

        customer_type varchar(255) check (customer_type in
('INDIVIDUAL','CORPORATE','PARTNERSHIP','TRUST','GOVERNMENT')),
        last_review_date date,
        next_review_date date,
        remarks varchar(500),
        risk_category varchar(255),
        customer_id bigint,
        primary key (id)
    )

```

Customers:

Specifications:

- id
- created_at
- created_by
- del_flg
- updated_at
- updated_by
- aadhar_number
- address_line1
- address_line2
- alternate_phone
- annual_income
- city
- country
- date_of_birth
- email

- first_name
- gender
- last_name
- middle_name
- nationality
- occupation
- pan_number
- phone_number
- pincode
- state
- status
- Primary Key(id)

DB Schema:

```

create table customers (
  id bigint generated by default as identity,
  created_at timestamp(6) not null,
  created_by varchar(255),
  del_flg boolean,
  updated_at timestamp(6),
  updated_by varchar(255),
  aadhar_number varchar(12),
  address_line1 varchar(255),
  address_line2 varchar(255),
  alternate_phone varchar(255),
  annual_income float(53),
  city varchar(255),
  country varchar(255),
  date_of_birth date,
  email varchar(255),

```

```

first_name varchar(255) not null,
gender varchar(255) check (gender in ('MALE','FEMALE','OTHER')),
last_name varchar(255) not null,
middle_name varchar(255),
nationality varchar(255),
occupation varchar(255),
pan_number varchar(10),
phone_number varchar(255) not null,
pincode varchar(6),
state varchar(255),
status          varchar(255)          check          (status          in
('PENDING','ACTIVE','INACTIVE','SUSPENDED','BLOCKED','REJECTED','CLOSE
D')),
    primary key (id)
)

```

KYC DETAILS:

Specifications:

- id
- created_at
- created_by
- del_flg
- updated_at
- updated_by
- address_proof_document_path
- address_proof_number

- address_proof_type
- id_proof_document_path
- id_proof_expiry_date
- id_proof_issue_date
- id_proof_number
- id_proof_type
- kyc_remarks
- pan_document_path
- pan_number
- photo_path
- signature_path
- verified_by
- verified_date
- customer_id
- Primary Key(id)

DB Schema:

```
create table kyc_details (
    id bigint generated by default as identity,
    created_at timestamp(6) not null,
    created_by varchar(255),
    del_flg boolean,
    updated_at timestamp(6),
    updated_by varchar(255),
    address_proof_document_path varchar(255),
    address_proof_number varchar(255),
```

```

        address_proof_type      varchar(255)      check      (address_proof_type      in
('AADHAAR','PASSPORT','UTILITY_BILL','BANK_STATEMENT','RENTAL_AGR
EEMENT','VOTER_ID')),
        id_proof_document_path varchar(255),
        id_proof_expiry_date date,
        id_proof_issue_date date,
        id_proof_number varchar(255),
        id_proof_type           varchar(255)       check      (id_proof_type           in
('AADHAAR','PASSPORT','DRIVING_LICENSE','VOTER_ID','NATIONAL_ID')),
        kyc_remarks varchar(255),
        pan_document_path varchar(255),
        pan_number varchar(255),
        photo_path varchar(255),
        signature_path varchar(255),
        verified_by varchar(255),
        verified_date date,
        customer_id bigint not null,
        primary key (id)
    )

```

user_roles:

Specifications:

- id
- name
- Primary Key(id)

DB Schema:


```
create table roles (  
    id bigint not null,  
    name varchar(255),  
    primary key (id)  
)
```

Users:

Specifications:

- id
- password
- username
- Primary Key(id)

DB Schema:

```
create table users (  
    id bigint not null,  
    password varchar(255),  
    username varchar(255),  
    primary key (id)  
)
```

Payments-service:

The payment requested are stored and transaction ID is generated , and notification is Send to user. For now these payment are pulled and proceed by external core banking system. And also same for notification ,The stored notification will be pulled and published by External system.

[illegible]

```
create table payments (
    transaction_id bigint generated by default as identity,
```

```

created_at timestamp(6) not null,
created_by varchar(255),
del_flg boolean,
updated_at timestamp(6),
updated_by varchar(255),
amount numeric(38,2),
bank_code varchar(255),
currency varchar(255),
description varchar(255),
exchange_rate numeric(38,2),
payment_type          varchar(255)          check      (payment_type          in
('INTERNATIONAL','DOMESTIC','WITHIN_BANK')),
receiver_account varchar(255),
receiver_name varchar(255),
reference_number varchar(255),
sender_account bigint,
sender_name varchar(255),
status varchar(255),
transaction_date timestamp(6),
transaction_fee numeric(38,2),
primary key (transaction_id)
)

```

Specifications:

- transaction_id
- created_at
- created_by
- del_flg
- updated_at

- updated_by
- amount
- bank_code
- currency
- description
- exchange_rate
- payment_type
- receiver_account
- receiver_name
- reference_number
- sender_account
- sender_name
- status
- transaction_date
- transaction_fee
- Primary Key(transaction_id)

Foreign Key Reference Table

Table	Foreign Key Column	References
Cif	customer_id	customers.id
Accounts	cif_id	cif.id
Addresses	customer_id	customers.id
kyc_details	customer_id	customers.id
user_roles	user_id	users.id
user_roles	role_id	roles.id

User Accounts Domain:

The customers table is the root entity. All core tables reference back to it.

customers → cif : One customer has one CIF record. The cif table holds the customer_id as a foreign key referencing customers.id

cif → accounts : One CIF can have multiple accounts. The accounts table holds the cif_id as a foreign key referencing cif.id

customers → addresses : One customer can have multiple addresses. The addresses table holds the customer_id as a foreign key referencing customers.id

customers → kyc_details : One customer has one KYC record. The kyc_details table holds the customer_id as a foreign key referencing customers.id.

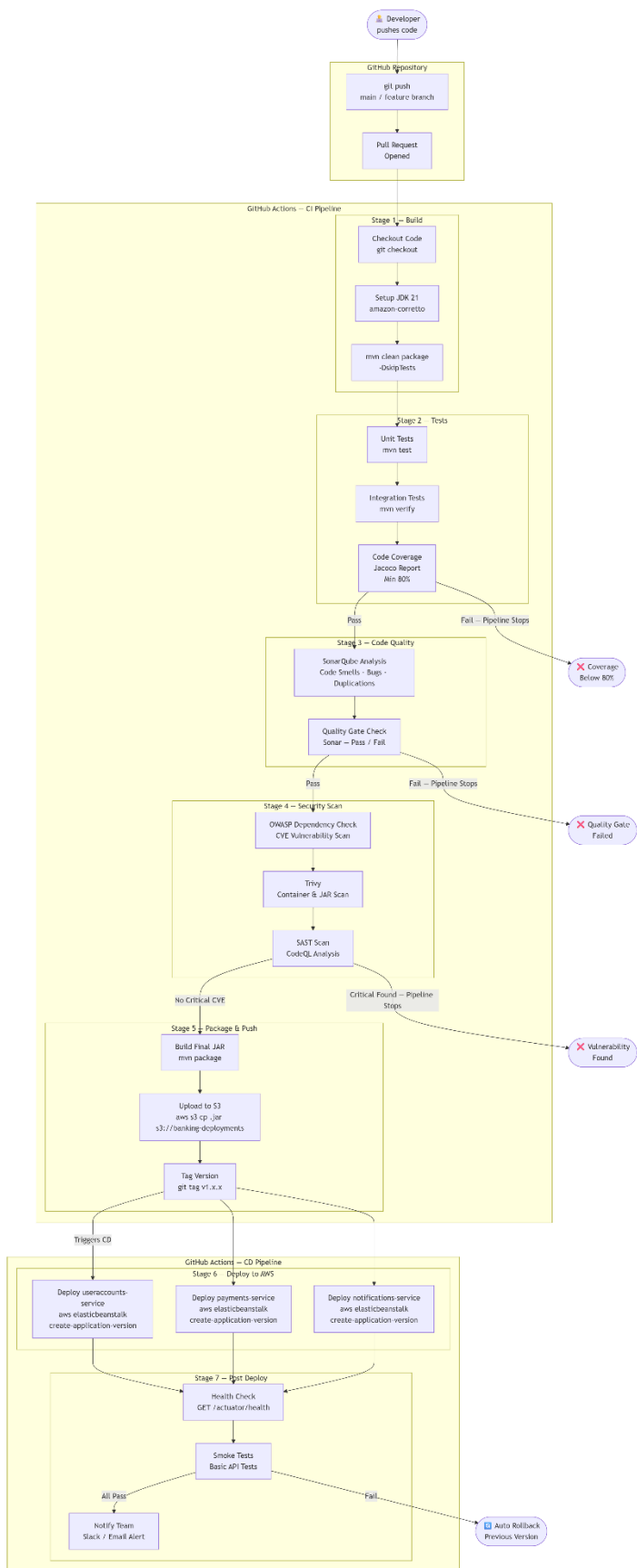
users → user_roles → roles : A user can have multiple roles and a role can be assigned to multiple users. This is a Many-to-Many relationship managed through the user_roles junction table. The user_roles table holds user_id referencing users.id and role_id referencing roles.id

Deployment Flow

CI/CD:

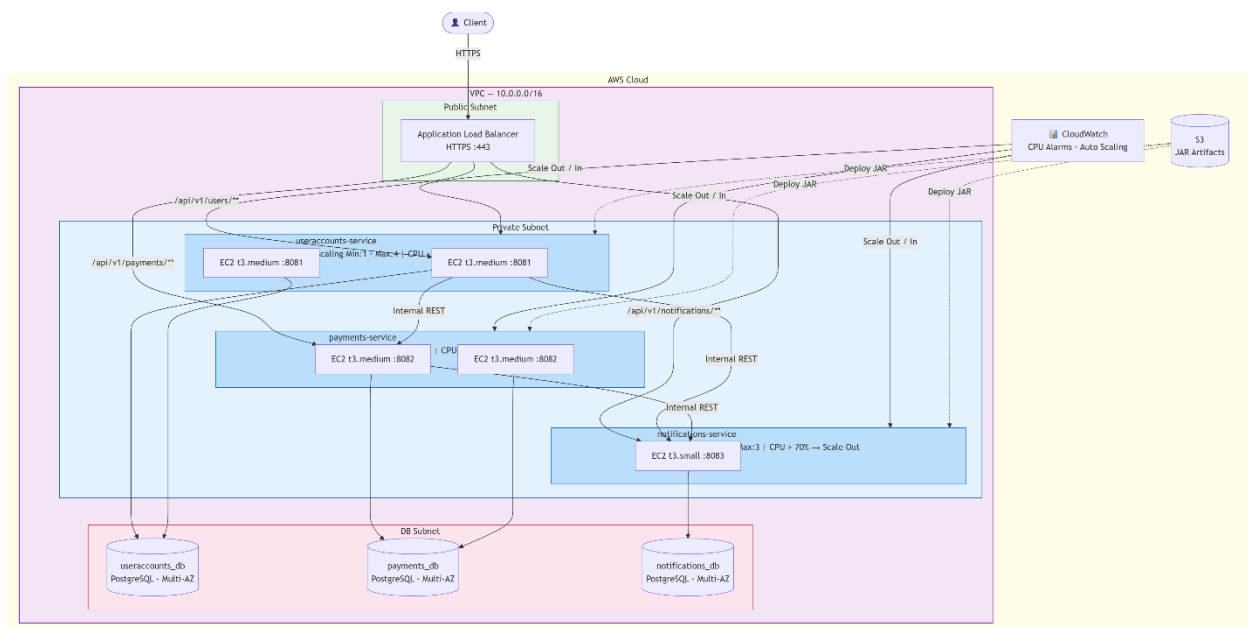
Source Code is maintained in Github, whenever new check in happens ie,code is pushed to main branch git hub actions pipe line triggers and it does below steps and once it success Image is pushed to s3 bucket,

Stage	What Happens
1 — Build	Checkout → JDK 21 → mvn clean package
2 — Tests	Unit tests → Integration tests → Jacoco coverage (min 80%)
3 — Code Quality	SonarQube analysis → Quality Gate (pass/fail blocks pipeline)
4 — Security Scan	OWASP CVE check → Trivy JAR scan → CodeQL SAST
5 — Package & Push	Final JAR → Upload to S3 → Git version tag
6 — Deploy	3 parallel Elastic Beanstalk deployments
7 — Post Deploy	/actuator/health → Smoke tests → Auto rollback if fail



Deployment:

- ALB receives HTTPS traffic and routes by URL path to each service
- 3 services in private subnet, each with Auto Scaling (min/max + CPU threshold shown inline)
- 3 RDS databases (one per service, Multi-AZ) in the DB subnet
- CloudWatch triggers scale out/in for all 3 Auto Scaling Groups
- S3 deploys JARs to each Elastic Beanstalk environment
- Inter-service REST calls between user accounts → payments → notifications



Performance optimization changes: (Autoscaling of Pods to handled peak volume):

Below Autoscaling startergy can be used particular in peak loads like salary processing and festival seasons

Auto Scaling automatically increases or decreases the number of EC2 instances running each microservice based on real-time traffic and CPU usage. Instead of running fixed

servers that either waste money during low traffic or crash during high traffic, the system dynamically adjusts capacity to match demand.

How It Works in This Banking System

Normal State (Low Traffic)

- Each service runs on a minimum of 1 EC2 instance
- CloudWatch continuously monitors CPU utilization every 60 seconds
- All 3 services — useraccounts, payments, notifications — handle requests on a single instance each

Scale Out (High Traffic)

- When CPU utilization crosses 60% for 2 consecutive minutes, CloudWatch fires an alarm
- The Auto Scaling Group automatically launches a new EC2 instance with the same service
- The new instance registers itself with the Application Load Balancer
- ALB immediately starts distributing incoming requests across both instances
- This process continues until CPU drops below threshold or maximum instances (4) is reached

Scale In (Low Traffic)

- When CPU drops below 30% for 5 consecutive minutes, CloudWatch fires a scale-in alarm
- Auto Scaling picks the least utilized instance for termination
- ALB stops sending new requests to that instance (connection draining — existing requests finish first)
- Instance is safely terminated with zero dropped requests
- A cooldown period of 120 seconds prevents rapid scale in/out cycles

Per Service Configuration

Service	Min	Max	Scale Out	Scale In	Why
useraccounts-service	1	4	CPU > 60%	CPU < 30%	Handles heavy KYC + customer onboarding logic
payments-service	1	4	CPU > 60%	CPU < 30%	Payment processing is compute intensive
notifications-service	1	3	CPU > 70%	CPU < 40%	Lighter workload — only sends email/SMS

Performance Benefits:

Handles Traffic Spikes Automatically:

During peak banking hours (salary credit days, month-end), payment and account service traffic spikes sharply. Auto Scaling adds instances within 2–3 minutes without any manual intervention, ensuring no degradation in response time.

Eliminates Single Point of Failure:

With multiple instances running behind the ALB, if one EC2 instance becomes unhealthy, the ALB health check detects it within 30 seconds and stops routing traffic to it. The remaining instances absorb the load with no downtime.

Cost Optimization:

During off-peak hours (midnight to 6 AM), traffic is minimal. Auto Scaling scales in to 1 instance per service, reducing EC2 costs significantly while maintaining availability.

Independent Scaling Per Service

Each microservice scales independently based on its own load. If payment traffic is high but notification traffic is low, only the payments-service scales out. This avoids unnecessary resource provisioning.

Zero Downtime Deployments

During new JAR deployments via Elastic Beanstalk, Auto Scaling performs a rolling update — replacing instances one at a time while keeping the remaining instances serving live traffic. Users experience no interruption.

Consistent Response Time Under Load

By keeping CPU utilization around 60%, each instance always has headroom to handle request bursts. This directly maintains low API response times even as concurrent users increase.

Technologies Used

Spring Boot

The core framework powering all three microservices — useraccounts-service, payments-service, and notifications-service. Handles REST endpoints, security, database access, and async processing.

AWS

Cloud platform where the three services are deployed and hosted. Provides the infrastructure (servers, networking, managed databases) to run the banking system in production.

GitHub

Version control platform where the banking system codebase is stored. Manages code history, branching, and collaboration across the three services.

GitHub Actions

CI/CD pipeline that automatically builds, tests, and deploys the services whenever code is pushed to the repository.

Maven

Build tool managing the project as a multi-module structure — one parent pom.xml with three child modules (payments-service, useraccounts-service, notifications-service). Handles dependency management and packaging each service into a runnable JAR.

Docker

Packages each Spring Boot service into a container image so it runs consistently across local development, testing, and AWS deployment environments.

SQL (MySQL)

Relational database storing all persistent data — payment transactions, user accounts, KYC details, and notifications. H2 in-memory database is used locally for development; MySQL is used in production.

Postman

API testing tool used to manually test all REST endpoints across the three services — sending requests, passing JWT tokens, and verifying responses during development.

IntelliJ IDEA

IDE used to write, navigate, debug, and run the Spring Boot services locally during development.

Conclusion:

Microservice Architecture :

Building the banking platform as three independent services — useraccounts-service, payments-service, and notifications-service — demonstrated how large banking systems separate responsibilities into focused, independently deployable units. Each service owns its domain, its own database, and communicates with others only through REST APIs, just like real-world digital banking backends.

Channel Banking Concept:

This project acts as a channel layer between the UI/Mobile interface and the Core Banking System — handling user-facing operations like registration, KYC, and payment initiation without replacing the core processing engine. This is how all modern Indian banks (HDFC NetBanking, SBI YONO, Axis Mobile) are architected.

Regulatory Compliance in Design:

Integrating PAN and Aadhaar as identity verification fields, building a KYC approval workflow with admin sign-off, and enforcing JWT authentication on every API reflects real RBI compliance requirements — teaching how software design decisions are driven by regulatory constraints, not just technical ones.

Security as a First Principle:

JWT tokens are issued at login by useraccounts-service and validated independently in every service through dedicated JwtAuthFilter classes. The token propagates from payments-service → accounts-service → notifications-service, securing the entire request chain without repeated authentication.

MVP-Driven Development:

Scoping the project to a Minimum Viable Product — user registration, account generation, payment initiation, and notification — taught how real product teams prioritise a working

core over a feature-complete system. This is the foundation all production banking platforms are built on before iterative releases.

Automated Delivery Pipeline:

The 7-stage GitHub Actions CI/CD pipeline — build, test, code coverage, SonarQube quality gate, OWASP vulnerability scan, deployment to AWS, and health verification — taught the full software delivery lifecycle from a code push to a live production deployment.

Practical Applications:

Channel Banking	Acts as the bridge between mobile/web UI and Core Banking, handling all
Payment Initiation	Payments-service accepts the transaction request, validates the sender balance with Core Banking System to process
KYC & Aadhaar/PAN Verification	Customer registration requires PAN and Aadhaar submission, with admin approval

Functional Limitations:

Payment Processing is Initiation Only:

The payments-service accepts the payment request, validates the balance, and stores the transaction with status PENDING. It does not process the payment in real time — the actual debit/credit is expected to be handled by the Core Banking System. As the project grows, real-time payment processing (NEFT, RTGS, IMPS, UPI) needs to be integrated. Beneficiary's are not maintained locally ,It is send to Payment processing system and get

Validated. And also IFSC lookup and SWIFT CODE look up is not enabled.

Notifications are stored not Published:

The notifications-service saves notification records to its database but does not actually deliver SMS or Email. The readme acknowledges this: *"Currently notifications are just logged to console. Need to integrate with actual email/SMS providers."* AWS SNS and Twilio integration are identified as future additions.

KYC Document Storage:

KYC details store document paths (idProofDocumentPath, panDocumentPath, photoPath) but there is no actual file upload or document storage mechanism implemented. In production, these would be stored in AWS S3 with secure access controls.

References:

Functional References:

#	Author / Organization	Title / Topic	Book / Paper / Report Name	Year
1	Infosys Finacle	Core Banking System Architecture	Finacle White Paper – Digital Banking Transformation in India	2018
2	TCS BaNCS	Channel Banking & Microservices in Indian Banks	TCS BaNCS White Paper – API-Led Core Banking	2018
3	NPCI	Retail Digital Payment Systems	NPCI Annual Report – Payment Systems in India	2018

Technical References:

#	Name of Website	Author	Title / Topic
1	Spring Official Docs	Spring Team	Spring Boot Reference Documentation
2	Spring Official Docs	Spring Team	Spring Security – JWT Authentication & Authorization
3	Spring Official Docs	Spring Team	Spring Data JPA – Repositories & Auditing
4	Baeldung	Eugen Paraschiv	Building REST API with Spring Boot
5	Baeldung	Eugen Paraschiv	Spring Boot Microservices – Inter-service Communication u

#	Name of Website	Author	Title / Topic
6	Baeldung	Eugen Paraschiv	JWT Authentication with Spring Security
7	Baeldung	Eugen Paraschiv	Spring Boot Email Notification – JavaMailSender

Tools used:

- Draw.io
- Notepad ++
- IntelliJ idea
- Word doc
- Mermaid editor/Lucid chart
- Maven
- Java 21
- Sql(H2 DB)
- Spring initializer
- Postman
- Swagger UI

